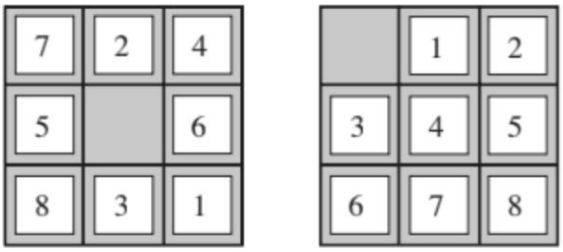
ARTIFICIAL INTELLIGENCE

Assignemnt 2

Jay Vankar

U24AI042

Q1 The following figure shows the start state and goal state for the 8-puzzle problem. Find the number of states explored before reaching the goal state using the BFS algorithm.



Start State Goal State

Code:

```
# Generate all possible next states
def generate_states(state):
    states = []
    zero_pos = state.index(0)

    # Move UP
    if zero_pos not in [0, 1, 2]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos - 3] = \
            new_state[zero_pos - 3], new_state[zero_pos]
        states.append(new_state)

    # Move LEFT
    if zero_pos not in [0, 3, 6]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos - 1] = \
            new_state[zero_pos - 1], new_state[zero_pos]
        states.append(new_state)
```

```
    # Move DOWN
    if zero_pos not in [6, 7, 8]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos + 3] = \
            new_state[zero_pos + 3], new_state[zero_pos]
        states.append(new_state)

    # Move RIGHT
    if zero_pos not in [2, 5, 8]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos + 1] = \
            new_state[zero_pos + 1], new_state[zero_pos]
        states.append(new_state)

    return states

# BFS
def bfs(initial_state, goal_state):
    open_list = [initial_state]   #queue
    visited = set([tuple(initial_state)])
    states_explored = 0

    while open_list:
        current_state = open_list.pop(0)

        if current_state == goal_state:
            print("BFS Goal Found")
            print("BFS States Explored:", states_explored)
            return
        states_explored += 1

        for next_state in generate_states(current_state):
            if tuple(next_state) not in visited:
                visited.add(tuple(next_state))
```

```
            open_list.append(next_state)

    print("Goal not found using BFS")

#MAIN
initial_state = [7, 2, 4,
                 5, 0, 6,
                 8, 3, 1]

goal_state = [0, 1, 2,
              3, 4, 5,
              6, 7, 8]

bfs(initial_state, goal_state)
```

Output:

```
PS C:\Users\Jay> python -u "d:\SEM4\AI\LAB\Assignment2\Q1.py"
BFS Goal Found
BFS States Explored: 168883
PS C:\Users\Jay>
```

Q2 Find the number of states explored for the DFS algorithm. Give comparison of BFS and DFS in terms of cost if the path cost for the state is given as depth of the search tree at which the state is occurring.

Code:

```
# Generate all possible next states
def generate_states(state):
    states = []
    zero_pos = state.index(0)

    # Move UP
    if zero_pos not in [0, 1, 2]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos - 3] = \
            new_state[zero_pos - 3], new_state[zero_pos]
        states.append(new_state)
```

```
    # Move LEFT
    if zero_pos not in [0, 3, 6]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos - 1] = \
            new_state[zero_pos - 1], new_state[zero_pos]
        states.append(new_state)

    # Move DOWN
    if zero_pos not in [6, 7, 8]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos + 3] = \
            new_state[zero_pos + 3], new_state[zero_pos]
        states.append(new_state)

    # Move RIGHT
    if zero_pos not in [2, 5, 8]:
        new_state = state[:]
        new_state[zero_pos], new_state[zero_pos + 1] = \
            new_state[zero_pos + 1], new_state[zero_pos]
        states.append(new_state)

    return states

# DFS
def dfs(initial_state, goal_state):
    stack = [initial_state]
    visited = set([tuple(initial_state)])
    states_explored = 0

    while stack:
        current_state = stack.pop()

        if current_state == goal_state:
            print("DFS Goal Found")
```

```
            print("DFS States Explored:", states_explored)
            return
        states_explored += 1

        for next_state in generate_states(current_state):
            if tuple(next_state) not in visited:
                visited.add(tuple(next_state))
                stack.append(next_state)

    print("Goal not found using DFS")

#MAIN
initial_state = [7, 2, 4,
                 5, 0, 6,
                 8, 3, 1]

goal_state = [0, 1, 2,
              3, 4, 5,
              6, 7, 8]

dfs(initial_state, goal_state)
```

Output:

```
PS D:\SEM4\AI\LAB\Assignment2> python -u "d:\SEM4\AI\LAB\Assignment2\Q2.py"
DFS Goal Found
DFS States Explored: 177786
PS D:\SEM4\AI\LAB\Assignment2>
```